Project Proposal:
Analysis of Parallelization Techniques on K-mer Counting for Genomic Sequences

Alex McGinn

September 13, 2026

1 Introduction

In genomic sequence analysis, K-mer counting consists of creating substrings for varying k-lengths from a biological sequence. These sequences consist of the four bases of DNA (A,C,G, and T), which we can count and use to compare DNA samples. They are called k-mers because in chemistry a "mer" is a repeating unit within a larger molecule (in genomics that molecule is mainly DNA). Larger values of k can provide more sequence specific information, while shorter values provide greater sensitivity[1]. Although, as k increases, the number of possible k-mers grows exponentially, leading to an increased amount of memory required to store and process k-mer counts. For large genomic datasets, these computations become computationally expensive, but there are various methods to accelerate this process using parallelization. The aim of this project is therefore to analyze the effectiveness of CPU/GPU parallelism for these types of tasks.

2 Background

The counting of K-mers is done by moving a sliding window of size k across the entire sequence and extracting every substring. This allows us to compare genomic sequences, without simply comparing the entire sequences. This is also useful because we can break a DNA sequence into many smaller subsequences of a known size. This also allows us to perform many of these computations in parallel, leading to a needed speedup. Although, as the input dataset size and value of k increase, the ability to store the data necessary in main memory becomes more difficult. One strategy for storing k-mers is to index an array or a similar data structure. This allows us to predict the amount of main memory necessary to store all possible combinations for a dataset and set k-mer value. For example, for some dataset, we could have an array that contains all possible k-mers for k-value set equal to five. We know whether this would be possible or not because the number of possible combinations of such a string with 4 possible bases is:

$$4^k$$

e.g. in the 5 k-mer example above:

$$4^5 = 1024 \text{ possible subsequences}$$

We could then also have an array that holds the count of each k-mer in the dataset. Therefore, we know that we need 1024 representations of the four possible bases. We can then also predict the size of the array used to count those occurrences. To determine the size necessary, we then need to consider the maximum possible number of occurences a k-mer can have. This of course will be dictated by frequency, which we cannot outright predict. However, we can be certain of the total number of possible occurrences based on the dataset size. That is because the maximum value will always be equal to how many possible substrings there are, determined by:

$$n - k + 1$$

e.g. in a DNA Sequence of length 66,000 with k-value equal to 5:

$$66000 - 5 + 1 = 65996$$

Which means that we need array elements that go up to at least 65996. In this instance, we would know we now need elements larger than an unsigned short, which would reach capacity at 65,535

3 Project Objectives

The objective of this project is to analyze the effects of parallelization on k-mer counting for genomic sequences. An initial implementation will be written in C++ performing the k-mer count for a baseline. The algorithm will then be written using OpenMP for parallelization on multiple CPU cores. Another implementation will utilize CUDA for GPU parallelization. MPI may also be evaluated if suitable distributed computing resources are available. There is a possibility of using an old desktop and Amazon EC2 instances to provide this additional resource.

Experiments will vary the dataset size and k-mer size. The performance will then be evaluated in standard methods. Most notably in speedup compared to the serial implementation, memory consumption, and the behavior of the implementation when scaling the dataset and k-mer size.

References

[1] John A. Lees, Minna Vehkala, Niko Välimäki, et al. Sequence element enrichment analysis to determine the genetic basis of bacterial phenotypes. *Nature Communications*, 7:12797, 2016.